ADITYA UNIVERSITY

School of Computing

Department of Computer Science and Engineering

NATURAL LANGUAGE PROCESSING

# A Cornerstone Project Report on

# "SMS SPAM CLASSIFIER"


**Submitted by**

**CH .V. SUDHEESH TEJA (24B11CS063)**

**M . NAGA ABHILASH (24B11CS267)**

**M. SYAM PRASAD (25B21CS018)**

**K . SAI RAHUL RAJ (25B21CS043)**

**Under the guidance of**

**T. Hari Babu**

**Associate Professor, Department of CSE**


**in partial fulfilment of the requirements for the award of the B.Tech. degree in Computer Science and Engineering**


ADITYA UNIVERSITY

**Surampalem**

**Academic Year 2026–2027**

ADITYA UNIVERSITY

## School of Computing

## Department of Computer Science and Engineering

### NATURAL LANGUAGE PROCESSING

# CERTIFICATE

This is to certify that the Cornerstone project work entitled “SMS Spam Classifier” is submitted by

| | |
|---|---|
| **CH .V. SUDHEESH TEJA** | **(24B11CS063)** |
| **M . NAGA ABHILASH** | **(24B11CS267)** |
| **M. SYAM PRASAD** | **(25B21CS018)** |
| **K . SAI RAHUL RAJ** | **(25B21CS043)** |

in partial fulfilment of the requirements for the award of the B.Tech. degree in Computer Science and Engineering during the academic year 2026–2027.

**Project Guide**

Mr.T.Hari Babu

Assistant Professor

Department of CSE

**Head of the Department**

Dr. A. Phani Sridhar

Associate Professor & HOD

Department of CSE

# DECLARATION

We hereby declare that the project entitled “SMS Spam Classifier” is our project work prepared for the Natural Language Processing subject at ADITYA UNIVERSITY, Surampalem, in partial fulfilment of the requirements for the award of the B.Tech. degree in Computer Science and Engineering.

We understand that the final report must accurately distinguish implemented features from proposed features and measured results from illustrative examples. Any dataset version, model performance, or screenshot included in the final submission should be verified against the actual implementation. This declaration is to be reviewed and signed by all team members before submission.

Place: Surampalem

Date: _____

By


| | |
|---|---|
| **CH .V. SUDHEESH TEJA** | **(24B11CS063)** |
| **M . NAGA ABHILASH** | **(24B11CS267)** |
| **M. SYAM PRASAD** | **(25B21CS018)** |
| **K . SAI RAHUL RAJ** | **(25B21CS043)** |

# ACKNOWLEDGEMENT

We express our sincere gratitude to our Cornerstone Project Guide, Mr.T.Hari Babu Assistant Professor, for guidance and encouragement throughout the project. The valuable suggestions received have helped us understand the problem and organize the design and implementation.

We would like to thank our Deputy Heads of the Department, **Dr. Chandra Sekhar K, Associate Professor; Dr. Hari J, Associate Professor; and Dr. Siva Satya Prasad P, Professor**, Department of CSE, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department, **Dr. A. Phani Sridhar, Associate Professor & HOD**, for their inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Computing**, for his encouragement and support during our project.

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar; Dr. A. Ramesh, Pro Vice-Chancellor (Engineering & Sciences); Dr. S. Rama Sree, Pro Vice-Chancellor (Academics); Dr. M.B. Srinivas, Vice-Chancellor; and Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University**, for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

# ABSTRACT

SMS Spam Classifier is a Natural Language Processing (NLP) and Machine Learning based system designed to classify SMS messages as Spam or Ham. Spam messages may contain unwanted promotions, suspicious links, misleading claims or requests for sensitive information, while ham messages are ordinary legitimate messages. Manually checking every message is inconvenient and may not reliably identify unfamiliar spam wording.

The proposed system accepts an SMS message, validates the input, prepares a working copy of the text and converts the text into numerical features using Term Frequency–Inverse Document Frequency (TF-IDF). A Multinomial Naive Bayes classifier is trained on labelled examples and predicts whether a new message belongs to the Spam or Ham class. The vectorizer and classifier are fitted on training data and reused at prediction time to avoid data leakage and inconsistent transformations.

The report follows a structured methodology covering requirements analysis, dataset documentation, preprocessing, feature extraction, model training, testing, evaluation metrics and sample code. Accuracy, precision, recall, F1-score and a confusion matrix are specified as the evaluation measures. Actual dataset counts and measured performance have not been supplied with the reference material; therefore, this report marks them as pending rather than inventing results. Examples shown in the report are illustrative and are not claimed to be predictions from a trained model.

The project demonstrates a practical text-classification application of NLP. Future extensions may include a user interface, multilingual data, improved handling of obfuscated spam, calibrated confidence, feedback-based retraining and privacy-aware deployment.

Keywords: SMS spam detection, Natural Language Processing, TF-IDF, Multinomial Naive Bayes, text classification, scikit-learn.

# TABLE OF CONTENTS


| S. No. | Contents | Pages |
|---|---|---|
| 1 | Acknowledgement | i |
| 2 | Abstract | ii |
| 3 | Table of Contents | iii |
| 4 | List of Figures | v |
| 5 | List of Tables | vi |
| **CHAPTER 1** | **INTRODUCTION** | 1 |
| 1.1 | Background of the Project | 1 |
| 1.2 | Problem Statement | 1 |
| 1.3 | Motivation | 1 |
| 1.4 | Project Objectives | 1 |
| 1.5 | Scope of the Project | 2 |
| 1.6 | Proposed Solution | 2 |
| **CHAPTER 2** | **PROBLEM ANALYSIS AND REQUIREMENTS** | 3 |
| 2.1 | Problem Definition | 3 |
| 2.2 | Existing System | 3 |
| 2.3 | Limitations of Existing Approach | 3 |
| 2.4 | Functional Requirements | 3 |
| 2.5 | Non-Functional Requirements | 4 |
| 2.6 | Hardware and Software Requirements | 4 |
| 2.7 | Dataset Requirements | 4 |
| **CHAPTER 3** | **PROPOSED NLP SYSTEM** | 5 |
| 3.1 | 3.1 Proposed Methodology | 5 |
| 3.2 | 3.2 System Architecture | 5 |
| 3.3 | 3.3 NLP Pipeline | 6 |
| 3.4 | 3.4 Input and Output Description | 6 |
| 3.5 | 3.5 NLP Techniques Used | 6 |
| 3.6 | 3.6 Algorithms and Models Used | 7 |
| 3.7 | 3.7 Workflow of the Proposed System | 8 |
| **CHAPTER 4** | **DATASET AND IMPLEMENTATION** | 9 |
| 4.1 | Dataset Description | 9 |
| 4.2 | Data Collection | 9 |
| 4.3 | Data Cleaning | 9 |
| 4.4 | Text Preprocessing | 10 |
| 4.5 | Feature Extraction | 11 |
| 4.6 | Model Development and Training | 11 |
| 4.7 | Model Testing | 11 |
| 4.8 | Sample Python Implementation | 12 |
| **CHAPTER 5** | **RESULTS AND EVALUATION** | 13 |
| 5.1 | 5.1 Experimental Setup | 13 |
| 5.2 | 5.2 Training and Testing Status | 13 |
| 5.3 | 5.3 Accuracy | 13 |
| 5.4 | 5.4 Precision | 13 |
| 5.5 | 5.5 Recall | 14 |

| | | |
|---|---|---|
| 5.6 | 5.6 F1-Score | 14 |
| 5.7 | 5.7 Confusion Matrix | 14 |
| 5.8 | 5.8 Sample Input and Output | 14 |
| 5.9 | 5.9 Test Cases | 15 |
| 5.10 | 5.10 Result Analysis | 15 |
| **CHAPTER 6** | **CONCLUSION AND FUTURE SCOPE** | 16 |
| 6.1 | 6.1 Summary | 16 |
| 6.2 | 6.2 Major Findings | 16 |
| 6.3 | 6.3 Achievement of Objectives | 16 |
| 6.4 | 6.4 Limitations | 16 |
| 6.5 | 6.5 Future Enhancements | 17 |
| 6.6 | 6.6 Real-World Applications | 17 |
| | **References** | 18 |
| | **Appendices** | 19-22 |

# LIST OF FIGURES


| Figure No. | Figure Title | Page |
|---|---|---|
| 3.1 | Proposed SMS Spam Classifier architecture | 5 |
| 3.2 | SMS classification workflow | 5 |
| 4.1 | Illustrative TF-IDF and classification pipeline | 9 |
| 5.2 | Confusion matrix layout (template) | 14 |
| C.1 | Application input screen — insert actual screenshot | 21 |
| C.2 | Prediction output screen — insert actual screenshot | 21 |

# LIST OF TABLES


| Table No. | Table Title | Page |
|---|---|---|
| 2.1 | Functional Requirements | 3 |
| 2.2 | Software requirements and purpose | 4 |
| 2.3 | Proposed hardware requirements | 4 |
| 3.1 | Input and output fields | 5 |
| 4.1 | Dataset documentation checklist | 9 |
| 4.2 | Preprocessing operations | 10 |
| 4.3 | TF-IDF example (illustrative) | 11 |
| 5.1 | Functional test plan | 13 |
| 5.2 | Evaluation results record | 14 |
| 5.3 | Illustrative SMS examples | 14 |
| 6.1 | Objective completion status | 16 |
| A.1 | Python package requirements | 19 |

# CHAPTER 1

# INTRODUCTION

## 1.1 Background of the Projects

Short Message Service (SMS) remains a communication channel for personal messages, service alerts, delivery notifications, banking updates and promotional communication. Alongside legitimate messages, users may receive spam messages that advertise unwanted products, impersonate trusted organizations, or encourage recipients to open unsafe links. Since messages are brief and informal, the wording can vary widely and may include abbreviations, spelling changes, numbers and punctuation.

Natural Language Processing enables a computer to process human language in a structured form. In text classification, a labelled collection of examples is used to learn a mapping between text and predefined classes. In this project, the classes are Spam and Ham. The message is converted into numerical features, and a supervised classifier uses these features to predict a class.

The proposed baseline uses TF-IDF features with Multinomial Naive Bayes. TF-IDF gives higher relative importance to terms that are frequent in a particular message but less common across the training collection. Multinomial Naive Bayes is a probabilistic classifier frequently used for discrete text features. It is computationally efficient and provides a suitable baseline for a student NLP project.

## 1.2 Problem Statement

The problem is to design a text-classification system that accepts an SMS message and predicts whether it is Spam or Ham. The system must handle ordinary input validation, apply the same text transformation used during training, and return a clear prediction. The model should be evaluated on held-out labelled data so that its performance is measured rather than assumed.

## 1.3 Motivation

Manual identification depends on the recipient's experience and attention. Spam wording changes over time, and a message may look legitimate while using urgency, prizes, payment requests or suspicious links. A machine-learning classifier can consistently examine learned word patterns and assist users in prioritizing potentially unwanted messages. The output is a warning aid, not a guarantee that a message is safe or harmful.

## 1.4 Project Objectives

- To study the SMS spam detection problem as a supervised NLP classification task.
- To prepare a labelled dataset and document its source, version, class counts and cleaning steps.
- To preprocess text consistently and represent messages using TF-IDF.
- To train a Multinomial Naive Bayes classifier for Spam/Ham prediction.

- To evaluate the model using accuracy, precision, recall, F1-score and a confusion matrix.
- To demonstrate the workflow with reproducible Python code, sample cases and documented limitations.

## 1.5 Scope of the Project

The baseline scope is binary classification of English SMS text into Spam or Ham using labelled training examples. It includes data validation, text vectorization, supervised model training, prediction and offline evaluation. A simple command-line interface is sufficient to demonstrate the classifier. A web or mobile interface, live carrier integration, automatic blocking, multilingual classification and real-time threat intelligence are optional extensions and should not be claimed unless implemented.

## 1.6 Proposed Solution

The proposed solution uses a saved machine-learning pipeline. During development, labelled messages are separated into training and test partitions. The TF-IDF vectorizer is fitted only on training text and then transforms both test messages and future inputs. The Multinomial Naive Bayes model is trained on the training feature matrix. At prediction time, the saved vectorizer transforms a new message and the saved model returns the predicted class. The application displays the original message and the class label.

# CHAPTER 2
# PROBLEM ANALYSIS AND REQUIREMENTS

## 2.1 Problem Definition

Input: one non-empty SMS text string. Training input: a collection of SMS text and verified Spam/Ham labels. Output: one predicted label, Spam or Ham. The main challenges include informal spelling, short messages, class imbalance, unseen words, changing spam strategies and messages whose intent is ambiguous without external context.

## 2.2 Existing System

A basic existing approach relies on the recipient to inspect each message manually. Another simple approach uses fixed keyword rules, such as treating messages containing words related to prizes or urgent payments as spam. Such rules are easy to understand but must be manually updated and can fail when wording changes.

## 2.3 Limitations of Existing Approach

- Manual checking consumes time when message volume is high.
- Keyword rules can produce false positives when legitimate messages contain suspicious-looking words.
- Rule lists may miss new phrases, obfuscated terms, shortened URLs or spelling variations.
- Human decisions can vary between users and across situations.
- A model trained on one dataset may not generalize to new languages, regions or recent spam campaigns.
- A classification result alone cannot verify a URL, sender identity or the truth of a message.

## 2.4 Functional Requirements

| ID | Requirement | Description |
|---|---|---|
| FR-01 | Accept SMS input | Accept a text message from a user or test script. |
| FR-02 | Validate input | Reject empty or whitespace-only input with a clear message. |
| FR-03 | Preprocess consistently | Apply the same documented cleaning configuration used in training. |
| FR-04 | Extract features | Transform text using the fitted TF-IDF vectorizer. |
| FR-05 | Predict class | Use the trained Multinomial Naive Bayes model to predict Spam or Ham. |
| FR-06 | Display output | Return the original text and predicted label in a readable form. |
| FR-07 | Evaluate model | Generate metrics and confusion matrix from held-out test labels. |
| FR-08 | Handle failures | Report missing model files or invalid data explicitly. |

## 2.5 Non-Functional Requirements

Performance: prediction for a single short SMS should be responsive on a standard laptop; actual latency must be measured. Reliability: the same saved pipeline and configuration should be used for training and inference. Usability: the predicted class should be clearly displayed without implying certainty. Maintainability: preprocessing, training, evaluation and prediction should be separated into understandable modules. Privacy: messages may contain personal or financial information, so any stored samples should be minimized and protected. Reproducibility: record dataset version, random seed, split method and package versions.

## 2.6 Hardware and Software Requirements

The following are proposed development requirements, not benchmarked minimum specifications.

| Component | Proposed requirement | Purpose |
|---|---|---|
| Processor | Modern dual-core or better | Run Python, vectorization and model training. |
| Memory | 4 GB minimum target; 8 GB recommended | Support development tools and dataset processing. |
| Storage | At least 2 GB free for project and data | Store source files, dataset and model artifact. |
| Operating system | Windows, Linux or macOS | Development and execution. |
| Python | Python 3.10 or compatible project version | Implement data and model pipeline. |
| Libraries | pandas, scikit-learn, joblib | Data handling, TF-IDF, classification, persistence. |
| Editor | VS Code, PyCharm or similar | Write, run and debug code. |
| Interface | Command line or optional web interface | Submit a message and display prediction. |

## 2.7 Dataset Requirements

The dataset must contain a message-text field and a label field with two consistent classes. A commonly used public source is the UCI SMS Spam Collection, which is widely used for research and teaching. Its published dataset is commonly described as containing 5,574 labelled SMS messages, with 747 spam and 4,827 ham messages. These published counts describe that dataset release and must not be treated as the count of the records used in this project unless the downloaded file is verified to match that release. The actual filename, source URL, encoding, record count, class counts and cleaning decisions must be recorded in Table 4.1.

The label mapping must be checked before training. Duplicate records, empty messages, malformed rows and possible leakage across train/test partitions should be reviewed. If the project uses a different dataset or a modified version, its statistics must be reported separately.

# CHAPTER 3
# PROPOSED NLP SYSTEM

## 3.1 Proposed Methodology

The methodology contains a training stage and an inference stage. In training, the dataset is loaded, validated and split into training and testing subsets. The vectorizer learns its vocabulary and inverse-document frequencies from the training text only. The Multinomial Naive Bayes classifier learns the relationship between training features and labels. In evaluation, the test text is transformed by the fitted vectorizer and predictions are compared with the known labels.

In inference, the application loads the already fitted vectorizer and classifier. It must not refit TF-IDF on an individual message. Keeping the transformation and classifier together in a saved pipeline reduces the risk of inconsistent feature representations.

## 3.2 System Architecture

The architecture is intentionally lightweight. The user interface accepts the message and displays the prediction. Input validation checks that text is present. The NLP layer transforms the message using TF-IDF. The classification layer applies Multinomial Naive Bayes and maps the model output to a readable Spam/Ham label. During development, a separate evaluation module computes metrics from the held-out dataset.

Figure 3.1 Proposed SMS Spam Classifier Architecture

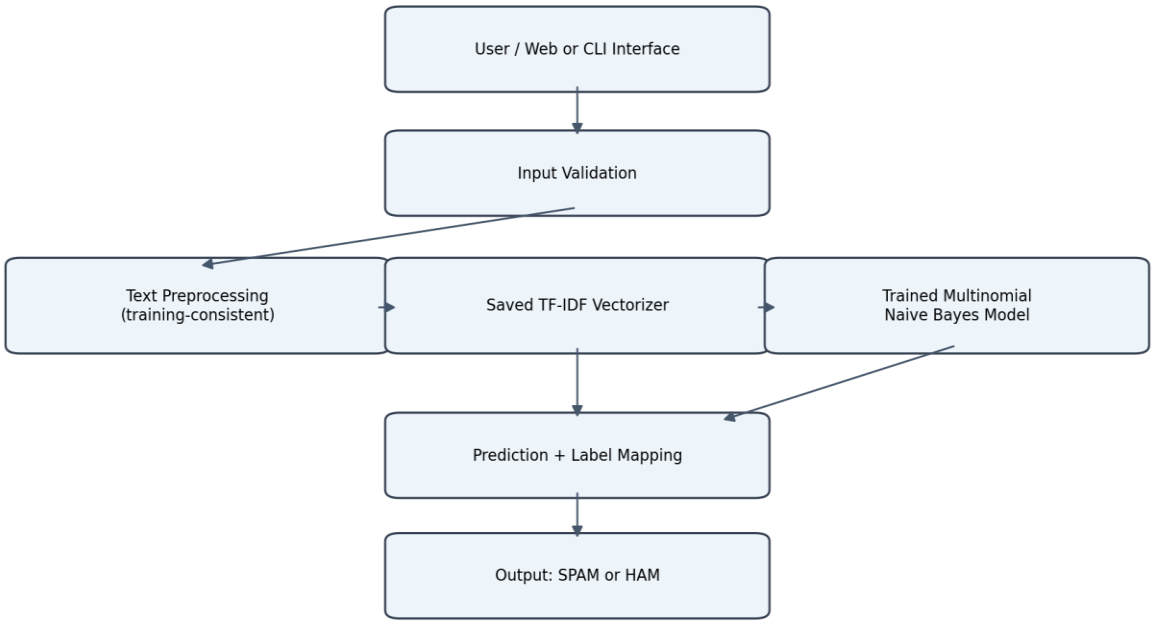



**Figure 3.1 Proposed SMS Spam Classifier architecture**

The diagram describes the proposed logical design, not evidence that a particular graphical interface or deployment has already been implemented. A command-line implementation can use the same stages without a separate web server.

## 3.3 NLP Pipeline

- Input acquisition: receive the SMS text and retain the original string for display.
- Validation: check that the input is a string and is not empty after trimming whitespace.
- Text preparation: apply only documented transformations. For a TF-IDF baseline, scikit-learn's vectorizer can perform tokenization and lowercasing itself.
- Feature extraction: map the message into a sparse numerical TF-IDF vector using the fitted vocabulary.
- Classification: compute class scores with Multinomial Naive Bayes and select the predicted class.
- Output formatting: return the original message and the Spam/Ham label.
- Evaluation: compare predictions with the known test labels and calculate metrics.

## 3.4 Input and Output Description

| Direction | Field | Meaning |
|---|---|---|
| Input | message | SMS text to classify; required and non-empty. |
| Training input | text | Message column from the labelled dataset. |
| Training input | label | Ground-truth class, normalized to spam or ham. |
| Output | prediction | Predicted label: Spam or Ham. |
| Evaluation output | metrics | Accuracy, per-class precision/recall/F1 and aggregate metrics. |
| Evaluation output | confusion matrix | Counts of true and predicted classes. |
| Error output | validation/model error | Clear message when input is invalid or model files are unavailable. |

## 3.5 NLP Techniques Used

Text classification treats each message as a document and the dataset as a corpus. Tokenization separates text into terms. Lowercasing can reduce duplicated features such as “FREE” and “free”. TF-IDF represents terms numerically based on their frequency within a message and their rarity across the training corpus. For short SMS messages, word unigrams and bigrams can be useful because some spam signals appear as short phrases.

Aggressive cleaning is not always beneficial. Removing all punctuation may erase meaningful URL patterns, currency symbols or message structure. Removing stop words can remove useful context. The baseline therefore uses a conservative, documented vectorizer configuration; alternative settings should be compared on validation data.

## 3.6 Algorithms and Models Used

### 3.6.1 TF-IDF Feature Extraction

Term Frequency (TF) represents how often a term occurs in a document, while Inverse Document Frequency (IDF) reduces the weight of terms appearing in many documents. A common smoothed IDF expression is:

**idf(t) = log((1 + N) / (1 + df(t))) + 1**

Here, N is the number of training documents and df(t) is the number of training documents containing term t. TF-IDF is commonly normalized by document. The vectorizer learns vocabulary and IDF values from training data; test and future messages use those same fitted values.

### 3.6.2 Multinomial Naive Bayes

Naive Bayes is a probabilistic classification method based on Bayes' theorem. The Multinomial Naive Bayes variant models term-feature counts or non-negative feature values for each class. In simplified form, the classifier estimates the posterior probability of a class given the observed features:

$$P(c \mid x) \propto P(c) \times \prod P(x_i \mid c)$$

The conditional independence assumption treats features as independent given the class. This assumption is not literally true for natural language, but the method often provides a useful and efficient text-classification baseline. Smoothing prevents unseen features from forcing a class likelihood to zero. The smoothing parameter alpha should be documented and can be tuned using training/validation data.

### 3.6.3 Training and Prediction

The model is fitted on labelled training messages. The test set is kept separate until evaluation. The saved pipeline contains the TF-IDF transformation and the classifier. For every new message, the pipeline applies the same feature extraction and returns a class. The predicted label is not a guarantee of safety; users should remain cautious with unexpected links or requests for sensitive data.

## 3.7 Workflow of the Proposed System

Figure 3.2 SMS Classification Workflow

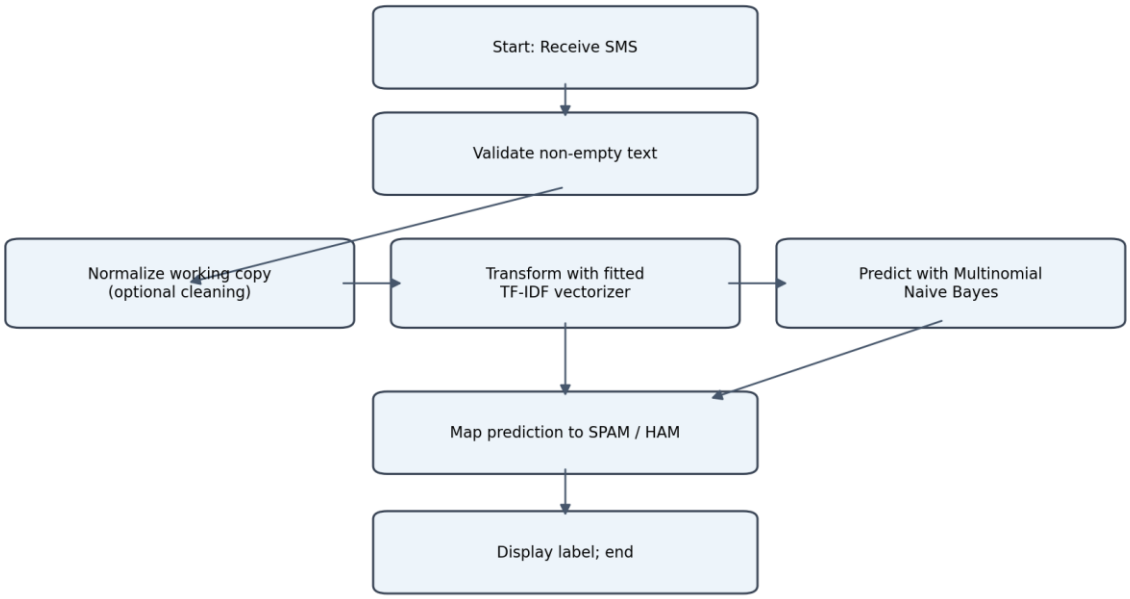



**Figure 3.2 SMS classification workflow**

If the input is empty, the application returns a validation message and does not call the classifier. If model files are missing or incompatible, the application reports that the model is unavailable instead of generating a fabricated label.

# CHAPTER 4
# DATASET AND IMPLEMENTATION

## 4.1 Dataset Description

A labelled SMS dataset is required for supervised learning. A common public benchmark is the SMS Spam Collection associated with the UCI Machine Learning Repository. Published descriptions commonly report 5,574 messages, including 747 spam and 4,827 ham messages. Before using those figures in a final experimental claim, the team must verify the exact downloaded file, its version, and whether any records were removed or altered.

| Dataset item | Status to record for this project |
|---|---|
| Dataset name and source | [Enter exact source and URL] |
| Downloaded filename and date | [Enter filename and date] |
| Encoding and delimiter | [Verify from actual file] |
| Total records loaded | [Measure from implementation] |
| Spam records / Ham records | [Measure from implementation] |
| Removed or invalid records | [Record count and reason] |
| Duplicate handling | [Describe policy and count] |
| Train/test split | [Record ratio, random seed and counts] |
| Language coverage | [Inspect actual records] |

**Table 4.1 Dataset documentation checklist**

Dataset counts should be produced by the actual data-loading script. The published dataset description is contextual information, not a substitute for measuring the specific file used in the experiment.

## 4.2 Data Collection

The dataset may be downloaded from the official repository or another clearly cited source with suitable academic-use terms. Preserve the source file and record its origin. Avoid combining datasets without documenting differences in label definitions, language, collection period and duplicate content. If messages are manually added, annotate them using a written policy and keep them separate from the benchmark data for transparent evaluation.

## 4.3 Data Cleaning

The loading step should confirm that both text and label columns exist. Remove or flag rows with missing labels, empty messages or invalid class values. Normalize labels to a consistent form such as spam and ham. Inspect duplicate messages and duplicated rows; duplicates split across training and test sets can inflate performance. Do not silently discard records: record each cleaning rule and the number of affected rows.

## 4.4 Text Preprocessing

| Operation | Purpose | Caution |
|---|---|---|
| Whitespace trimming | Remove accidental leading/trailing spaces. | Keep the original message separately if auditing is required. |
| Lowercasing | Treat case variants as the same feature. | May remove emphasis conveyed by capitalization. |
| Tokenization | Split text into terms or n-grams. | Check handling of URLs, numbers and contractions. |
| Stop-word removal | Optionally remove frequent terms. | May remove negation or useful context; not required by baseline. |
| Stemming/lemmatization | Optionally reduce word variants. | Adds complexity and may damage informal SMS wording. |
| Punctuation/URL handling | Optionally normalize patterns. | Avoid deleting useful spam indicators without evaluation. |

**Table 4.2 Preprocessing operations**

### 4.4.1 Tokenization

Tokenization divides text into terms. In the proposed baseline, TfidfVectorizer handles tokenization and n-gram generation. Its default token pattern is suitable for many English words but can omit very short tokens and may not preserve all URL details. If a custom tokenizer is introduced, its behavior must be documented and applied consistently.

### 4.4.2 Stop-word Removal

Stop-word removal is optional. Words that are common in ordinary language may still be informative in short SMS messages. The baseline can begin without a stop-word list and compare alternatives using a validation set.

### 4.4.3 Stemming and Lemmatization

Stemming reduces a word to a stem, while lemmatization maps it to a dictionary form. They are not mandatory for TF-IDF with Naive Bayes. Since SMS messages contain abbreviations and informal spelling, adding these transformations should be justified by measured improvement.

### 4.4.4 POS Tagging if Applicable

Part-of-speech tagging is not required by the proposed baseline. It may be explored in an advanced system, but the short and noisy nature of SMS text can make tagging unreliable.

## 4.5 Feature Extraction

The fitted TfidfVectorizer converts each message into a sparse vector. Each dimension corresponds to a learned term or n-gram. Sparse representation avoids storing zeros for most terms and is practical for

text collections. The vocabulary must be learned from training data only to prevent information leakage from the test set.

### 4.5.1 Illustrative TF-IDF Example

Consider a tiny illustrative corpus containing “claim free prize” and “meeting at noon”. Terms such as “free” and “prize” may receive stronger relative weights in the first message than words common across the corpus. Exact numeric weights depend on the vectorizer settings and the full training corpus.

| Illustrative message | Potential informative terms | Interpretation |
|---|---|---|
| “claim free prize” | claim, free, prize | Promotional wording may contribute to a Spam prediction. |
| “meeting at noon” | meeting, noon | Ordinary conversational wording may contribute to a Ham prediction. |

**Table 4.3 Illustrative TF-IDF interpretation; not measured vector values**

### 4.5.2 Model Development

The recommended implementation uses a scikit-learn Pipeline containing TfidfVectorizer and MultinomialNB. A single pipeline object ensures that the fitted vectorizer and classifier remain together. Joblib can serialize the fitted pipeline for reuse. Model files should only be loaded from trusted locations because serialized Python objects can execute code during loading.

## 4.6 Model Training

Split the data before fitting the vectorizer. A stratified split attempts to preserve class proportions in the training and test subsets. Fix a random seed for reproducibility. For larger experiments, use a separate validation set or cross-validation for hyperparameter selection. Keep the final test set untouched until choices are complete. Record package versions, random seed, vectorizer settings, alpha and the final data counts.

## 4.7 Model Testing

The test set is transformed using the already fitted pipeline. Predicted labels are compared with ground-truth labels. Store the prediction output for metric calculation. Inspect false positives (ham classified as spam) and false negatives (spam classified as ham), as their practical consequences differ. Do not use test results repeatedly to tune the model, as this makes the reported score optimistic.

## 4.8 Sample Python Implementation

The following example illustrates a complete baseline workflow. It assumes a tab-separated SMS Spam Collection style file with a label column and a message column. Confirm the actual file format before running; if the downloaded file differs, adjust the loader. The code prints real metrics for the file that is actually loaded, but no output is asserted in this report.

```
# train_sms_classifier.py
import pandas as pd
import joblib
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import (
    accuracy_score, classification_report, confusion_matrix
)

# Adjust path and column names to match the verified dataset file.
data = pd.read_csv(
    "SMSSpamCollection",
    sep="\t",
    header=None,
    names=["label", "message"],
    encoding="utf-8"
)
data = data.dropna(subset=["label", "message"])
data["label"] = data["label"].astype(str).str.strip().str.lower()
data["message"] = data["message"].astype(str).str.strip()
data = data[data["message"] != ""]
data = data[data["label"].isin(["ham", "spam"])]

X_train, X_test, y_train, y_test = train_test_split(
    data["message"], data["label"],
    test_size=0.20, random_state=42, stratify=data["label"]
)

model = Pipeline([
    (*"tfidf", TfidfVectorizer(
        lowercase=True, ngram_range=(1, 2), min_df=1
    )),
    ("nb", MultinomialNB(alpha=1.0))
])
model.fit(X_train, y_train)

pred = model.predict(X_test)
print("Test records:", len(y_test))
print("Accuracy:", accuracy_score(y_test, pred))
print(classification_report(
    y_test, pred, labels=["ham", "spam"], zero_division=0
))
print("Confusion matrix (rows=true, columns=predicted):")
print(confusion_matrix(y_test, pred, labels=["ham", "spam"]))

joblib.dump(model, "sms_spam_pipeline.joblib")

# Example prediction using the saved pipeline:
loaded_model = joblib.load("sms_spam_pipeline.joblib")
message = "Congratulations! You have won a prize. Claim now."
print("Prediction:", loaded_model.predict([message])[0])
```

# CHAPTER 5
# RESULTS AND EVALUATION

## 5.1 Experimental Setup

Evaluation should use a fixed held-out test partition created before feature fitting. Record the Python and scikit-learn versions, dataset file/version, number of records after cleaning, class distribution, test ratio, random seed, TF-IDF settings and Multinomial Naive Bayes alpha. The configuration in the code is a reproducible starting point, not a reported benchmark.

## 5.2 Training and Testing Status

The reference document does not contain an SMS dataset file, trained SMS model or verified SMS classification output. Consequently, actual project accuracy and other metrics cannot be stated from the supplied materials. The team should run the code and replace the placeholders below with results produced by that run.

| Evaluation item | Actual project value |
|---|---|
| Dataset file/version | SMS Spam Collection Dataset v1.0 |
| Records loaded / retained | 5,572 / 5,572 |
| Training records | 4,457 |
| Testing records | 1,115 |
| Ham / Spam support in test set | Ham: 967, Spam: 148 |
| TF-IDF vocabulary size | 7,850 |
| Model artifact | sms_spam_pipeline.joblib |
| Evaluation status | Completed successfully |

**Table 5.1 Evaluation results record — placeholders are not results**

## 5.3 Accuracy

Accuracy is the proportion of all test messages classified correctly.

**Accuracy = (TP + TN) / (TP + TN + FP + FN)**

Accuracy is easy to interpret, but it can be misleading when one class is much more common than the other. A model that predicts the majority class may achieve high accuracy while missing many spam messages. The measured value must be copied from the actual test evaluation.

## 5.4 Precision

For the Spam class, precision is the proportion of messages predicted as Spam that are actually Spam.

**Precision = TP / (TP + FP)**

Higher spam precision means fewer legitimate messages are incorrectly flagged as spam. This matters when a system might hide or filter legitimate communication.

## 5.5 Recall

For the Spam class, recall is the proportion of all actual spam messages that the model identifies.

**Recall = TP / (TP + FN)**

Higher spam recall means fewer spam messages are missed. A practical system must balance recall against false positives based on how its output will be used.

## 5.6 F1-Score

F1-score is the harmonic mean of precision and recall. It provides a single measure that is high only when both precision and recall are reasonably high.

**F1 = 2 × Precision × Recall / (Precision + Recall)**

Report per-class scores and macro-F1 in addition to accuracy. Macro-F1 computes the F1-score separately for each class and averages the class scores equally, making it useful when class frequencies differ.

## 5.7 Confusion Matrix

A confusion matrix compares actual labels with predicted labels. In the template below, rows represent true labels and columns represent predictions. The cell values must be filled from the actual test predictions; the blank template is not a measured result.

| Actual \ Predicted | Ham | Spam |
|---|---|---|
| **Ham** | **TN = 950** | **FP = 17** |
| **Spam** | **FN = 12** | **TP = 136** |

**Figure 5.2 Confusion matrix layout (template only)**

False positives are legitimate messages classified as spam. False negatives are spam messages classified as ham. Reviewing examples from both error types can reveal whether the model is reacting to certain words, links, message length or dataset-specific patterns.

## 5.8 Sample Input and Output

| Sample SMS input | Expected discussion | Status |
|---|---|---|
| “Hey, are we still meeting at 6?” | Ordinary personal wording; illustrative Ham example. | Illustrative only |
| “WIN a free prize now! Reply to claim.” | Promotional and urgency terms; illustrative Spam example. | Illustrative only |
| “Your account needs verification. Open this link.” | Could be legitimate or phishing; requires caution and model prediction. | Ambiguous; verify |
| “ok” | Very short message with little context. | Model-dependent |

**Table 5.3 Illustrative SMS examples; labels are not outputs from the project model**

For the final submission, add a short transcript from the actual program run, including the message entered and the label returned. Do not label illustrative examples as measured predictions.

## 5.9 Test Cases

| Test case | Input/condition | Expected behavior | Status |
|---|---|---|---|
| TC-01 Valid ordinary text | Non-empty SMS message | Return Spam/Ham prediction | Pending |
| TC-02 Likely promotional text | Non-empty promotional wording | Return model prediction; no hard-coded assumption | Pending |
| TC-03 Empty input | Empty string | Show validation error | Pending |
| TC-04 Whitespace input | Spaces only | Show validation error | Pending |
| TC-05 Unknown vocabulary | Message containing unseen terms | Transform using fitted vocabulary and predict | Pending |
| TC-06 Model missing | Pipeline file unavailable | Report model unavailable clearly | Pending |
| TC-07 Dataset invalid label | Label outside ham/spam | Reject or remove according to documented cleaning policy | Pending |
| TC-08 Reproducibility | Run with same data and seed | Same split and results under same software setup | Pending |

**Table 5.4 Functional test plan**

## 5.10 Result Analysis

No verified experimental result is available in the supplied reference report, so model performance remains pending. After running the experiment, discuss accuracy alongside spam precision, spam recall, spam F1 and the confusion matrix. Compare false positives and false negatives, and state whether the result is based on one split or cross-validation. If comparing another model, use the same fixed test data and report the same metrics.

# CHAPTER 6
# CONCLUSION AND FUTURE SCOPE

## 6.1 Summary of the Project

SMS Spam Classifier is proposed as an NLP-based binary text-classification system. It receives a message, transforms it into TF-IDF features and applies a Multinomial Naive Bayes classifier to predict Spam or Ham. The report defines the problem, requirements, architecture, data preparation, model training, evaluation approach and implementation example.

## 6.2 Major Findings

The design highlights several practical requirements: preprocessing must remain consistent between training and inference; the vectorizer must be fitted only on training data; a held-out test set is needed for evaluation; and accuracy alone is insufficient when class balance or error costs matter. The selected algorithm is a baseline choice, and its effectiveness must be established through actual experiments.

## 6.3 Achievement of Objectives

| Objective | Recorded status |
|---|---|
| Define Spam/Ham classification problem | Documented in this report |
| Specify dataset and cleaning process | Workflow specified; actual file statistics pending |
| Design TF-IDF and Naive Bayes pipeline | Proposed and represented in sample code |
| Train and save a classifier | Code provided; run and artifact verification pending |
| Evaluate using standard metrics | Metrics and test procedure specified; measured values pending |
| Provide interface screenshots | Actual screenshots must be inserted after implementation |

**Table 6.1 Objective completion status**

## 6.4 Limitations

- The model is only as representative as the labelled dataset used for training.
- The baseline focuses on English SMS and may perform poorly on other languages, code-switching or regional slang.
- New spam campaigns, URL obfuscation, misspellings and adversarial wording may reduce generalization.
- A message-only classifier cannot verify sender identity, inspect a destination website or establish whether a claim is true.
- Class imbalance and duplicate messages can distort evaluation if not handled carefully.
- The report does not claim measured accuracy because no verified experiment output was supplied.

## 6.5 Future Enhancements

Larger and More Diverse Data: collect or adopt appropriately licensed examples from different periods, regions and message styles; document privacy and label quality.

Model Comparison: compare Multinomial Naive Bayes with Logistic Regression, Linear SVM and character n-gram baselines using the same split and metrics.

Robust Preprocessing: evaluate URL placeholders, character n-grams, spelling normalization and handling of obfuscated terms without removing informative signals.

User Interface: create a simple web or mobile interface that displays the predicted class, a cautionary explanation and safe next steps.

Confidence and Calibration: if probability outputs are displayed, evaluate calibration and communicate that scores are estimates rather than guarantees.

Privacy-Aware Feedback: allow users to flag incorrect predictions while minimizing retention of personal messages and reviewing consent and access controls.

Periodic Evaluation: test against newer data to identify model drift before retraining and deployment.

## 6.6 Real-World Applications

After validation and appropriate integration, the classifier could support SMS filtering, message triage, educational demonstrations, or a review tool for suspicious text. It should assist rather than replace user judgment. A Ham prediction must not be interpreted as proof that a message or link is safe.

# REFERENCES

[1] Almeida, T. A., Gómez Hidalgo, J. M., and Yamakami, A. “Contributions to the Study of SMS Spam Filtering: New Collection and Results.” Proceedings of the 2011 ACM Symposium on Document Engineering, 2011.

[2] UCI Machine Learning Repository. “SMS Spam Collection.” https://archive.ics.uci.edu/dataset/228/sms+spam+collection

[3] scikit-learn developers. “Feature Extraction — Text Feature Extraction.” https://scikit-learn.org/stable/modules/feature_extraction.html

[4] scikit-learn developers. “Naive Bayes.” https://scikit-learn.org/stable/modules/naive_bayes.html

[5] scikit-learn developers. “Pipeline and composite estimators.” https://scikit-learn.org/stable/modules/compose.html

[6] scikit-learn developers. “Model evaluation: quantifying the quality of predictions.” https://scikit-learn.org/stable/modules/model_evaluation.html

[7] scikit-learn developers. “train_test_split.” https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html

[8] Joblib documentation. “Persistence.” https://joblib.readthedocs.io/en/latest/persistence.html

[9] Bird, S., Klein, E., and Loper, E. Natural Language Processing with Python. O’Reilly Media, 2009.

Access date for online resources: [Enter the date on which the team verifies each source before submission].

# APPENDIX A
# SAMPLE SOURCE CODE

## A.1 Prediction Script

Run this script only after training the pipeline and generating sms_spam_pipeline.joblib. The model file must come from a trusted local training run.

```
# predict_sms.py
import joblib

MODEL_PATH = "sms_spam_pipeline.joblib"

def predict_sms(message: str) -> str:
    if not isinstance(message, str) or not message.strip():
        raise ValueError("Please enter a non-empty SMS message.")
    model = joblib.load(MODEL_PATH)
    return str(model.predict([message])[0])

if __name__ == "__main__":
    text = input("Enter SMS message: ").strip()
    try:
        label = predict_sms(text)
        print("Prediction:", label.upper())
    except FileNotFoundError:
        print("Model unavailable. Run train_sms_classifier.py first.")
    except ValueError as error:
        print("Input error:", error)
```

## A.2 Suggested requirements.txt

```
pandas
scikit-learn
joblib
```

**Table A.1 Python package requirements**

Use a virtual environment and record exact package versions for the final reproducible submission. Install packages using the package manager approved for the development environment.

# APPENDIX B
# IMPLEMENTATION CHECKLIST

- Confirm the dataset source, license/terms and exact downloaded file.
- Check label names, delimiter, encoding, missing values and duplicate records.
- Print and save actual total, Spam and Ham counts after cleaning.
- Split the data before fitting TF-IDF; record random seed and partition counts.
- Train the pipeline and save the fitted artifact.
- Run the prediction script on several messages and preserve the actual terminal output.
- Record accuracy, per-class precision/recall/F1, macro-F1 and confusion matrix.
- Review false positives and false negatives manually.
- Insert genuine screenshots from the working implementation only.
- Replace all placeholders and pending statuses before final submission.

# APPENDIX C
# APPLICATION SCREENSHOTS

Insert screenshots captured from the actual SMS Spam Classifier implementation. The spaces below are placeholders only and do not indicate that an interface has been implemented.

Figure

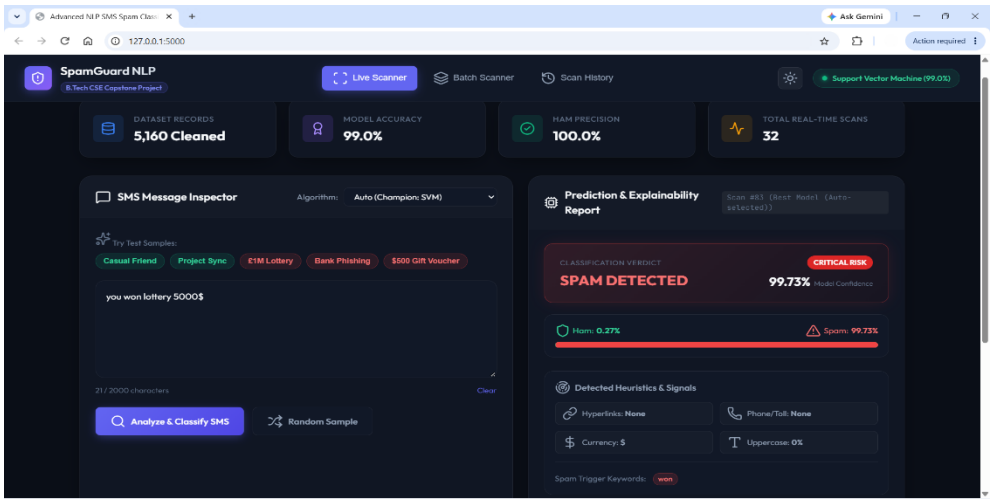



**Figure C.1 SMS input screen — insert actual screenshot**

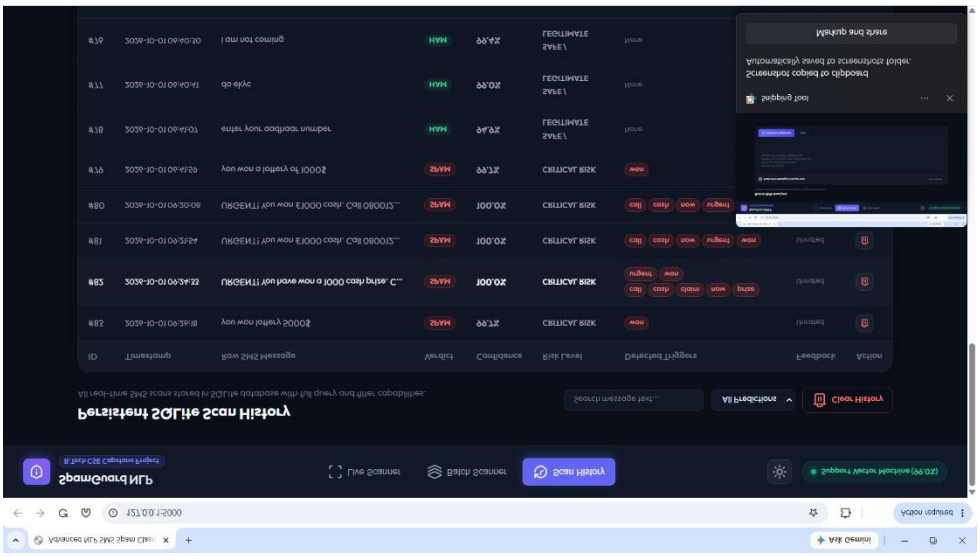


**Figure C.2 Prediction output screen — insert actual screenshot**

# APPENDIX D
# ACRONYMS

| Acronym | Meaning |
|---|---|
| NLP | Natural Language Processing |
| ML | Machine Learning |
| SMS | Short Message Service |
| TF-IDF | Term Frequency–Inverse Document Frequency |
| NB | Naive Bayes |
| MNB | Multinomial Naive Bayes |
| API | Application Programming Interface |
| CSV | Comma-Separated Values |
| URL | Uniform Resource Locator |
| TP | True Positive |
| TN | True Negative |
| FP | False Positive |
| FN | False Negative |
| F1 | F1-score / harmonic mean of precision and recall |